

While it builds.

One spec, one line of prompt, and about an hour of silence. This is what is happening inside that hour, and why the spec is doing more work than the prompt.

> Read SPEC.md and build it.



Eight milestones, data layer first.

M0	Models, schema, seed, /api/state	14 min
M1	Four mutations, the error table	12 min
M2	CSS tokens, layout, tab state	10 min
M3	The assets table	12 min
M4	Search, filters, empty state	8 min
M5	Add-asset drawer, validation	12 min
M6	Assign, return, repair, retire	8 min
M7	Dashboard tiles, People tab	12 min

WHY THE DATA LAYER FIRST

M0 and M1 finish before a single component exists, so the whole data layer is exercisable from `curl` while there is still no UI to blame.

A bug found later is then either in the repository or in the UI, and never ambiguously both.

IT WILL NOT ASK YOU ANYTHING

The spec's last section tells it to pick the simplest reading, state the assumption in one line, and keep going. Those one-line assumptions are worth reading. Each is a place your spec was ambiguous.

Expect one line per milestone, no essays. The numbers are the spec's own estimates, not a promise.

Four builds. One spec. **One variable.**

SPEC.md is byte-identical on every branch · only .claude/ changes

- 1 **checkpoint-1** the reviewer only · build from the spec, then audit it
- 2 **checkpoint-2** + **api-boundary** · one skill decides what the spec left open
- 3 **checkpoint-3** + two finished apps · review code you did not write, no build
- 4 **checkpoint-4** + **csharp-quality**, **blazor-component-hygiene** · near zero findings

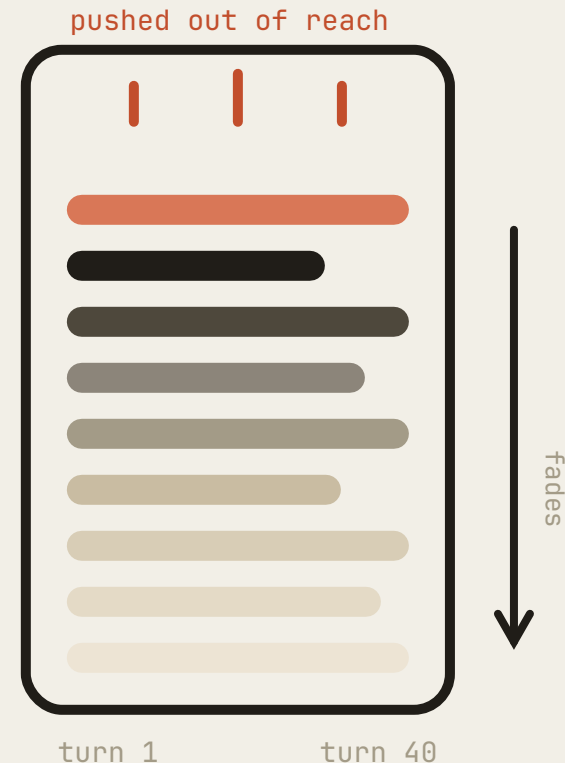
Same spec, same one-line prompt, every time. If the results differ, **the skills did it**, because nothing else moved. That is the entire experiment, and it only works because the spec never changes.

`ls .claude/skills` is the command that tells you which checkpoint you are actually on.

Context is not memory. It is a room filling up.

Everything in a session sits in **one finite window**: your prompts, its replies, every file it read, every command's output. Nothing is stored. It is all re-read, every single turn. As the room fills, the earliest things in it stop carrying weight.

- one heroic five-hour chat that slowly forgets its own decisions
- + durable facts in files, short sessions, fresh windows for fresh work



A whole ERP, laid on you **in one meeting.**

Someone walks you through procurement, then payroll, then inventory, then the approval matrix, then the tax rules, for four hours, with no notes taken. By hour three you are not stupid. You are **full**.

You start blending payroll's approval rules into procurement. You confidently repeat a number that was corrected an hour ago. You forget the constraint from minute ten entirely. **And you do not feel any of it happening.**

THE FOUR FAILURES, IN PEOPLE AND IN MODELS

Blending. Two rules from two subsystems merge into one that was never stated.

Staleness. Decision v1 is still in the room next to v3, and v1 sometimes wins.

Dropout. The constraint from early on stops being applied at all.

Confidence. None of it is announced. The answers stay fluent.

WHAT A GOOD ENGINEER DOES INSTEAD

Asks for it in writing. Splits it across sessions. Writes the decisions down where they can be re-read, instead of trusting a tired room to remember them.

Same mitigation for the model as for you: **write it down, keep the session short, re-read the document rather than the conversation.**

Five habits that keep the room empty.

1 • SEE IT BEFORE YOU FEEL IT

`/context` shows what is actually filling the window right now: memory files, tool output, the transcript. Run it once mid-build and the abstraction becomes a number.

2 • CLEAR BEATS COMPACT, WHEN YOU CAN

`/clear` starts an empty window; your `CLAUDE.md` and skills reload, so nothing durable is lost. `/compact` summarizes instead, which is cheaper than re-explaining. But a summary is **lossy**, and what it drops is silent.

3 • ONE SESSION, ONE PIECE OF WORK

Finish a milestone, commit, clear. A session that spans four unrelated tasks is carrying three tasks' worth of dead weight into the fourth.

4 • SEND A SUBAGENT

A subagent gets **its own window** and returns only its conclusion. Ten files get read; one paragraph comes back. That is how the reviewer on this branch works, and why it can afford to read the whole app.

5 • PUT DURABLE FACTS IN FILES, NOT IN CHAT

Anything you would have to retype next session belongs in a file: `CLAUDE.md`, a skill, or `SPEC.md`. A file is re-read at full weight every session. A sentence from turn six is not.

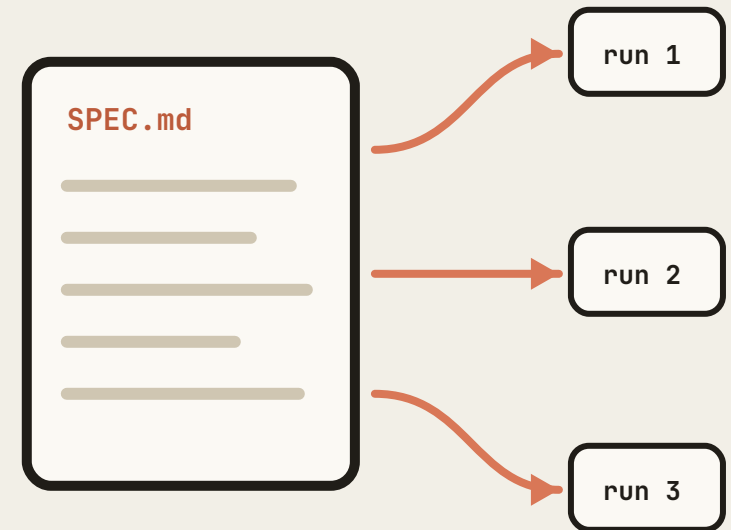
THE TELL

You re-explain something you already said, and the next answer still ignores it. Do not argue with it a third time; that adds to the pile. Clear, and put the fact in a file.

Longer sessions are not more knowledgeable. They are more crowded.

A document does not get tired.

`SPEC.md` is read at the start, at full weight, every run. It is the same 600 lines for every attendee in this room. Nobody's version drifted because their session went long, and nobody's version is better because they explained it more patiently.



same input, comparable output

- the requirements live in a conversation that is already fading
- + the requirements live in a file, in Git, reviewable in a pull request

Prompting is a chat. A spec is a **contract**.

Both can produce this app. Only one of them produces it **the same way twice**, survives the author leaving, and can be argued about before any code exists. The prompt you typed was eight words. All the engineering was already done, in a file, in advance.

prompting



every run a new path

spec driven



fixed checkpoints, one path

- "build me an asset tracker, use SQLite, oh and make it dense"
- + Read SPEC.md and build it.

Where each one actually wins.

	PROMPTING	SPEC DRIVEN
Run it twice	Two different apps	Two comparable apps
Reviewing the plan	After the code exists	Before, in a pull request
Handing it to a colleague	Retell the conversation	Send them the file
Auditing the result	Against taste and memory	Against a numbered clause
Small exploratory change	Faster, just ask	Overhead you do not need
You do not know what you want	Correct tool	Premature

Neither one is better. **Prompt to explore, spec to commit.** The mistake is staying in exploration mode for work that three other people will have to live with.

Ten sections, and one of them is a wall.

SPEC.md · AssetDesk v3.0

1

Goal

three questions the tool must answer fast

2

Non-goals

do not build these

3

Tech constraints

exact scaffold, exact packages

4

Data model

records, enums, invariants, schema, errors

5

Screens

behaviour + acceptance criteria per screen

6

Visual design

colour tokens, type scale, layout

7

Seed data

the exact 5 people and 12 assets

8

Build order

M0 to M7, done-when per milestone

9

Definition of done

twelve checkboxes

10

Working agreement

how to behave while building

THE TEST OF A SPEC

Not "is it detailed". The test is: **can two different engineers read it and disagree about what to build?** Every place they could is a place your agent will guess, silently and fluently.

WHAT MAKES SECTION 7 UNUSUAL

The seed data is spelled out to the individual serial number. That precision is what makes 5 employees, 12 assets a **verifiable fact** instead of a vibe, on every machine in this room.

WHAT IS DELIBERATELY ABSENT

No file-by-file instructions. No code to copy for the components. The spec constrains **outcomes and boundaries**, and leaves the typing to the agent.

The best section is the list of things to **not build**.

Left alone, a competent agent writes competent, idiomatic, **enterprise** .NET: interfaces over the repository, a service layer, AutoMapper, a Result wrapper, maybe Docker. All defensible. All wrong for a one-day prototype. Section 2 is what stops it.

~~Entity Framework~~
~~repository interface~~
~~service layer~~
~~AutoMapper~~
~~Result<T>~~
~~Tailwind~~
~~Docker~~
~~dark mode~~

§2

+ If something is not described in this document, do not build it.

A rule in prose is a hope. In SQL it is a wall.

- "an assigned asset should always have an owner"
- + `CHECK (status = 'assigned' AND assigned_to IS NOT NULL)`
OR ...

THE EIGHT INVARIANTS, AND WHO ENFORCES EACH

Database: unique tags, the assigned/unassigned pair, the enum value lists, the foreign key.

Repository: only in-stock assets can be assigned; retiring clears the assignment in the same transaction.

Nobody, by design: employees are never deleted, because no method exists to do it. The safest code is the code you did not write.

WHAT A CHECK CONSTRAINT BUYS YOU

Invariants 2 and 3 say an asset is either assigned **with** an owner and a date, or unassigned **with neither**. Written as prose, every future change has to remember it. Written as a CHECK, the database refuses. A bug that was possible became impossible.

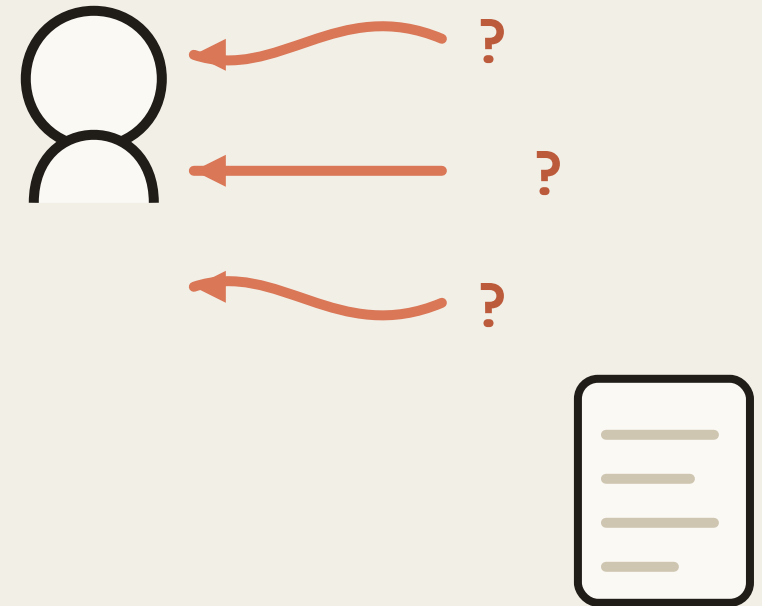
THE TRAP THE SPEC HAD TO CALL OUT

`PRAGMA foreign_keys` is **per connection**, and the driver pools connections, so setting it once at startup silently stops applying. It belongs in the connection string. Invariant 5 holds or fails on that one detail, and the failure is invisible.

Later today you will fire a `curl` with a fabricated employee id. If it is rejected, that connection string is right.

Do not ask for a spec. Let it **interview you.**

"Write me a spec for an asset tracker" returns something plausible, generic, and full of decisions you never made. The useful move is the opposite: dump what you know, then make it **find the holes** and ask you about them.



your answers are the spec

- "write a spec for an IT asset tool" → plausible, generic, not yours
- + "here is my brain dump. Ask me every question you would have to guess at."

Six moves, in this order.

1 · BRAIN DUMP, UNSTRUCTURED

Screens, rules, things you hate about the current tool. Do not organise it. Organising is the machine's job.

2 · MAKE IT ASK, NOT ANSWER

"Ask me every question you would otherwise have to guess at. Do not write the spec yet." Expect twenty questions. **The uncomfortable ones are the valuable ones:** they are the decisions you were postponing.

3 · ANSWER, AND SAY WHAT IS OUT OF SCOPE

Every "not now" you say becomes a non-goal. This is where section 2 gets written, and it is free.

4 · NOW HAVE IT DRAFT

Give it the shape you want: goal, non-goals, constraints, data model with invariants, screens with acceptance criteria, build order, definition of done.

5 · ATTACK THE DRAFT FROM A FRESH SESSION

A new window, no history: **"where is this ambiguous, and what would you have to invent?"** The session that wrote it cannot see its own gaps; it remembers what it meant. A cold reader only has the words.

6 · BUILD ONCE, THEN FIX THE SPEC

Every assumption it announced while building is a hole in the document. Patch the spec, not just the code. The spec is the artifact that runs again tomorrow.

Steps 2 and 5 do most of the work, and both are the same move: **let something that cannot read your mind tell you what your document does not say.**

600 lines, and it never says **how the UI gets its data.**

TWO DEFENSIBLE READINGS

A. Components call the repository directly. No HTTP, no serialisation, synchronous, simplest thing that works, and section 3.4 even praises it.

B. Components call the six `/api/*` endpoints the spec already requires. One boundary, testable from outside, the shape you would need if a mobile client ever arrived.

WATCH WHICH ONE YOURS PICKED

`grep -r AssetRepository AssetDesk/Components/`. Hits mean reading A, silence means reading B.

WHY NOT JUST ADD A PARAGRAPH

You could. But "the UI must go through the API" is not a fact about **this product**. It is a standard about **how your team builds any product**. Bury it in one app's spec and it dies with that app.

THAT IS WHAT CHECKPOINT 2 IS

The same spec, plus one skill that decides exactly this one thing. Then the same prompt, the same review, and we compare the counts. Nothing else moves.

A spec cannot be finished. It can only be **finished enough that the remaining guesses do not matter**, and this one does matter, which is the point.

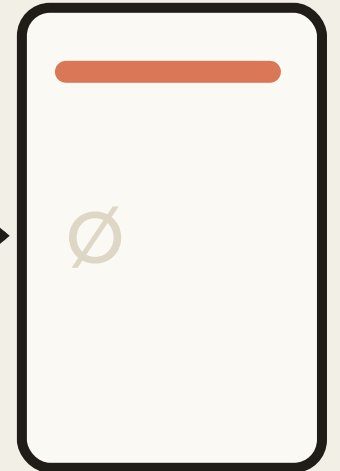
Never let it grade its own **homework**.

Ask the session that just wrote the code to review it, and it reviews **its own reasoning**. It already talked itself past every shortcut once. It will under-report, convincingly. So /review-build does not review anything itself. It dispatches somebody else.

wrote the code



reads it cold



no shared history
no memory of the shortcuts
read only: no write tools at all

- "now review what you just built" → a short, flattering list
- + a fresh agent, its own window, reads the code cold

A skill that only delegates.

```
.claude/skills/review-build/SKILL.md
---
name: review-build
description: Use when reviewing an AssetDesk build
  on this branch - dispatches a cold-context reviewer
---

Dispatch the assetdesk-reviewer agent.
Do not review the code yourself.

## Why you must not review it yourself
If you wrote this code earlier in the conversation,
reviewing it means reviewing your own reasoning,
and you will under-report.
```

WHAT THE REVIEWER READS

SPEC.md, then **every** SKILL.md on the branch except review-build itself. Those skills are binding rules, not suggestions.

THE LINE THAT MAKES THE EXPERIMENT WORK

"A branch with fewer skills has fewer rules. **That is the design, not a gap for you to fill.**" Without it the reviewer would import its own standards, every branch would score the same, and the comparison would die.

ONE AGENT PER DIRECTORY

On checkpoint 3 there are two apps. Two agents, two windows, so neither review contaminates the other.

Note what this skill is: **nine lines of instruction and a refusal.** Most of its value is in what it forbids.

Four numbers, comparable across every branch.

```
BOUNDARY:  n
QUALITY:   n
SPEC:      n
TOTAL:     n
```

Each finding lands in exactly one bucket. Where a finding could sit in two, the most specific rule wins and it is counted once.

THE REVIEWER'S OWN RULES • WHY THE NUMBER IS TRUSTWORTHY

Point at a line. A file and a line number, or it is not a finding. No "consider whether".

Do not pad. A zero bucket prints zero and says so plainly.

Absence is not a defect. Missing tests, missing interfaces, missing CI are all correct here, because section 2 forbids them. A reviewer that flags them is scoring the spec, not the build.

One decision, one finding. Five components injecting the repository is one finding with five locations. Otherwise architecture noise drowns everything else.

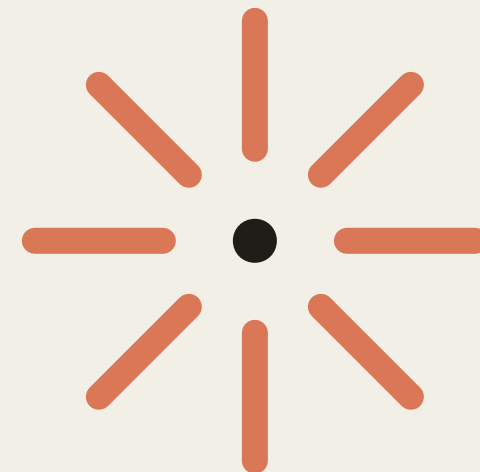
WHAT TO DO WITH TODAY'S NUMBER

Expect roughly ten findings. Sort them into two piles. **Architecture:** it chose a shape the spec left open. **Quality:** it wrote something a careful reviewer would reject. Checkpoints 2 and 4 remove those two piles **separately**.

Three commands, then **argue with it.**

```
# does it run, and did the seed actually seed
dotnet run --project AssetDesk --urls http://localhost:5198
curl -s localhost:5198/api/state | jq '{e:(.employees|length),a:
(.assets|length)}'
# → { "e": 5, "a": 12 }
```

/review-build



then: checkpoint 2 →

- + click a sidebar tab first: if nothing happens, the render mode is missing